Luv Kush Natural

Comprehensive Code Audit Report

Backend · Frontend · Database · Architecture · Security

Repository: `luvkush-old` · Branch: `kishan` · Commit: `a871b5c`
Stack: Angular 20 (SSR) · Node.js + Express + TypeScript · MySQL 8 · Razorpay · Shiprocket
Generated: 1 August 2026 · Findings are analysis only — no code was modified.

12	27	41	38	118
CRITICAL	HIGH	MEDIUM	LOW / QUALITY	TOTAL

Executive Summary

The codebase is **architecturally sound** — clean controller → service → SQL layering on the backend, fully lazy-loaded routes with signal-based state on the frontend, and generally correct SSR platform guards. No SQL injection was found; sort/order parameters are properly allow-listed and all queries use parameter binding. Passwords use bcrypt at cost 12, and refresh tokens are stored and rotated in the database.

However, the audit found **severe defects concentrated in the money path**. The payment flow trusts client-supplied amounts and order identifiers, meaning a customer can pay ₹1 for an order of any value, or replay a valid payment signature onto a different unpaid order. Razorpay webhook verification is broken in two independent ways simultaneously. Refunds have no upper bound. Separately, the coupon discount a customer sees at checkout is never actually transmitted to the server, so displayed and charged totals diverge.

On the infrastructure side, the deployment serves everything over **plain HTTP with no TLS**, while JWTs live in `localStorage`; and because `trust proxy` is never set behind nginx, all rate limiting collapses into a single shared bucket for the entire internet. One endpoint (`GET /users/profile`) queries a column that does not exist in the schema and will fail 100% of the time.

> **Immediate action, before anything else:** rotate the Razorpay key secret and the Shiprocket account password. Both are committed in plaintext at `backend/.env.example`, which is tracked in git and explicitly whitelisted by `.gitignore`. Also rotate `JWT_SECRET` — `backend/gen_tokens.js` is committed and hardcodes the shipped default secret while minting a working `super_admin` token.

Severity Distribution by Area

Area	Critical	High	Medium	Low	Dominant Theme
Payments & Orders	6	8	9	3	Client-trusted amounts; no transactions; broken webhook
Authentication & Session	2	7	6	4	Plaintext tokens; stale JWTs; broken refresh flow
Infrastructure & Config	3	2	5	6	No TLS; committed secrets; broken proxy trust
Database & Schema	1	3	8	7	Missing column; counter drift; missing indexes

Frontend	0	6	9	12	Silent error swallowing; contract mismatches
Code Quality	0	1	4	6	Dead code; duplication; zero test coverage

Critical Findings

LK-C01 — Customer controls how much they pay CRITICAL

`backend/src/controllers/payment.controller.ts:10,14 → services/payment.service.ts:32`

`advance_amount` is read directly from the request body and passed to `createRazorpayOrder()`, where `chargeAmount = advanceAmountOverride ?? order.total_amount`. It is never validated against the product's configured `advance_amount`, against a minimum, or against the order total.

Impact: A customer posts `{order_id: 5, advance_amount: 1}`, pays ₹1, and `/payment/verify` then marks the order `payment_status='paid'` and `status='confirmed'`. Any order of any value can be fully confirmed for ₹1. Direct, unbounded revenue loss.

LK-C02 — Payment signature replay across orders CRITICAL

`backend/src/services/payment.service.ts:60–92`

The HMAC signature is verified over `razorpay_order_id|razorpay_payment_id`, but the code never checks that `razorpay_order_id` matches the `razorpay_order_id` stored on the row identified by the client-supplied `order_id`. The `WHERE id = ? AND user_id = ?` clause proves ownership only.

Impact: Place a ₹100 order A and a ₹50,000 order B. Pay for A legitimately, capture the valid signature, then call `/payment/verify` with A's signature but `order_id = B`. The signature verifies and order B is marked paid. There is also no idempotency guard, so `/verify` can be replayed repeatedly.

LK-C03 — Webhook signature check is skipped when the secret is unset CRITICAL

`backend/src/services/payment.service.ts:110–120`

The guard reads `if (webhookSecret && expectedSig !== signature) throw`. `RAZORPAY_WEBHOOK_SECRET` appears nowhere in `.env.example`, in `utils/config.ts`, or anywhere else in the repository — so it is unset by default and the entire verification silently no-ops.

Impact: `POST /api/v1/payment/webhook` is unauthenticated. Anyone can send an unsigned `payment.captured` payload naming any `razorpay_order_id` and flip that order to `paid` / `confirmed` without paying anything.

LK-C04 — Webhook signature can never validate even once configured CRITICAL

`backend/src/app.ts:53 vs backend/src/routes/payment.routes.ts:10`

`express.json()` is registered globally at app level, before any route is mounted. By the time the request reaches the webhook route's `express.raw()`, body-parser has already consumed the stream and set `req._body`, so `raw()` is skipped and `req.body` is a parsed object. The service then computes the HMAC over `JSON.stringify(payload)` — a re-serialisation that will not byte-match Razorpay's original payload.

Impact: The webhook is unfixable by configuration alone. Setting the secret to close LK-C03 would cause every legitimate Razorpay webhook to be rejected with 400, silently breaking payment reconciliation. Both states are broken.

LK-C05 — Refunds have no upper bound CRITICAL

`backend/src/services/payment.service.ts:148–168`

`initiateRefund(orderId, amount)` forwards any client-supplied `amount` straight to Razorpay with no check against `order.total_amount`, against `advance_paid_amount`, or against amounts already refunded. The order is then unconditionally marked `refunded`.

Impact: Over-refund or repeated double-refund of the same payment. Admin-gated, so it requires a compromised or careless admin account — but there is no ledger check to catch it, and refunds are never written to `activity_logs`.

LK-C06 — Live third-party credentials committed to git CRITICAL

`backend/.env.example:34–39 (tracked; .gitignore:19 whitelists it)`

The file contains non-placeholder values: `RAZORPAY_KEY_SECRET`, and a real Shiprocket account email with a strong password. Default JWT secrets are also present.

Impact: Anyone with repository access — past collaborators, forks, or a future leak — can charge and refund through the Razorpay account and control shipping through Shiprocket. These credentials must be treated as compromised and rotated now; removing the file does not undo the git history.

LK-C07 — Committed script forges a super_admin token CRITICAL

`backend/gen_tokens.js:2–8 (tracked in git)`

The script hardcodes `'luvkush-jwt-secret-2026-change-in-prod'` — byte-identical to the `JWT_SECRET` default in `.env.example` — and signs a token with `role: 'super_admin'` for `userId: 1`.

Impact: If the deployed secret was never rotated (which the presence of this working script strongly suggests), anyone who reads the repo can mint a valid super-admin token and take over the entire admin panel. Full compromise.

LK-C08 — Production serves everything over plain HTTP CRITICAL

`backend/nginx.conf:9–10 (listen 80 only; no 443, no redirect)`

There is no TLS configuration and no HTTP→HTTPS redirect anywhere in the deployment config.

Impact: Login credentials, JWT access and refresh tokens, addresses, phone numbers and payment initiation traffic all travel in cleartext and are trivially interceptable on any shared network. Combined with token storage in `localStorage` (LK-H21) this is the highest-yield attack surface in the system. Razorpay checkout also requires HTTPS in production.

LK-C09 — Profile endpoint queries a column that does not exist CRITICAL

`backend/src/services/user.service.ts:8 and :25`

Both `getProfile()` and `updateProfile()` reference `avatar_url`. That column is absent from `schema.sql`, absent from `migrations/`, and never added by the boot-time migrations in `server.ts`. Verified by search across the whole backend.

Impact: `GET /api/v1/users/profile` and `/account/profile` throw `ER_BAD_FIELD_ERROR` and return 500 on every single call. The account profile page is permanently broken for all users unless the production database was manually altered outside version control.

LK-C10 — Stock oversell race; inventory can go negative CRITICAL

`backend/src/services/order.service.ts:22–30 (check) vs :89–96 (decrement)`

Availability is validated *before* `db.transaction()` opens, and the decrement `SET stock_quantity = stock_quantity - ?` carries no `WHERE stock_quantity >= ?` guard. `products.stock_quantity` is a signed `INT`, so negatives are accepted.

Impact: Two concurrent checkouts for the last unit both pass the check and both decrement — the classic TOCTOU oversell. Stock silently goes negative and the shop accepts orders it cannot fulfil. The same race applies to coupon `usage_limit` (validated outside, incremented inside the transaction).

LK-C11 — Rate limiting is globally broken behind nginx CRITICAL

`backend/src/app.ts:64–74 & routes/auth.routes.ts:10–14;` `trust proxy` `never set`

nginx proxies to Node and forwards `X-Forwarded-For`, but Express is never told to trust it (verified: no `trust proxy` anywhere in the backend). Every request therefore appears to originate from `127.0.0.1`.

Impact: All users share one rate-limit bucket. The auth limiter (`max: 10 / 15 min`) means ten total login attempts from anyone lock out the entire internet — a trivial denial of service. Conversely, per-IP brute-force protection does not exist at all. One misconfigured line negates every limiter in the application.

LK-C12 — Coupon discount is shown but never charged CRITICAL

`frontend/.../checkout.component.ts:151–159 → backend/src/services/order.service.ts:36–44`

The `placeOrder()` payload contains only `shipping_method`, `payment_method` and the address — `coupon_code` is never sent (confirmed by search across the checkout feature). Server-side, `if (data.coupon_code)` is therefore always false, and the surrounding `try/catch` swallows coupon failures silently anyway.

Impact: The customer sees a discounted total on the review screen and is then charged the full amount with no error and no explanation. Direct billing dispute and chargeback exposure. The entire coupon subsystem is non-functional end to end.

High Severity Findings

LK-H01 — Path traversal enables arbitrary file write HIGH

`backend/src/routes/media.routes.ts (folder from req.body → path.join → mkdirSync/toFile)`

`folder` is taken from the request body and joined onto the upload directory with no normalisation or containment check, then `fs.mkdirSync(..., {recursive:true})` creates it and sharp writes into it.

Impact: An admin (or anyone with a forged token per LK-C07) can pass `../../../../var/www/html` and write `.webp` files anywhere the Node process can write — including web roots and config directories.

LK-H02 — Production checkout cannot open Razorpay HIGH

`frontend/src/environments/environment.production.ts:5 → checkout.component.ts:199`

The checkout passes `key: environment.razorpayKeyId`, but the production environment file sets `razorpayKeyId: ''`. Meanwhile the backend already returns a correct `key_id` in the `/payment/create-order` response, which the frontend ignores.

Impact: Every online payment in a production build fails at gateway open. 100% of online checkout is broken in prod.

LK-H03 — Refresh-token endpoint URL mismatch HIGH

`frontend/.../auth.service.ts:100 calls /auth/refresh; backend route is /auth/refresh-token`

Impact: The refresh call always 404s. Sessions simply die when the 8-hour access token expires, with no silent renewal. Compounded by LK-H04 and by the absence of any HTTP interceptor to trigger a refresh on 401 — the method is effectively dead code.

LK-H04 — Refresh token DB lifetime contradicts its JWT lifetime HIGH

`backend/src/services/auth.service.ts:190 (hardcoded 7 days) vs .env.example:15 (JWT_REFRESH_EXPIRES=30d)`

`storeRefreshToken()` hardcodes a 7-day `expires_at` regardless of configuration.

Impact: The JWT remains cryptographically valid for 30 days but the database row expires at 7, so refresh fails from day 8 with a misleading "invalid or expired" error. Configuration silently does not do what it says.

LK-H05 — Changing your password does not end other sessions HIGH

`backend/src/services/user.service.ts:36–47`

`resetPassword()` correctly deletes all refresh tokens (auth.service.ts:150), but `changePassword()` does not.

Impact: A user who changes their password because they suspect compromise leaves every attacker session fully active. This is the exact scenario password change exists to solve.

LK-H06 — Review moderation is bypassed entirely HIGH

`backend/src/services/review.service.ts:60`

Reviews are inserted with a hardcoded `status = 'approved'`, even though the schema defaults to `'pending'` and a full admin moderation UI exists (`adminGetAll`, `updateStatus`).

Impact: Any logged-in user publishes directly to the storefront with no review. Spam, abuse and defamatory content go live instantly; the admin moderation queue is permanently empty and gives false assurance.

LK-H07 — "Helpful" votes write one column and read another HIGH

`review.service.ts:73 writes helpful_count ; :8 and :14 read and sort by helpful_votes`

Both columns exist in the schema (schema.sql:371–372), so no error is raised.

Impact: The feature is silently inert — votes are recorded into a column that is never displayed, while the UI shows and sorts by a column that is never written. Review ordering is permanently wrong and no one gets an error.

LK-H08 — Unlimited self-voting on reviews HIGH

`backend/src/services/review.service.ts:71–76 ( userId parameter accepted but unused)`

There is no per-user dedup table and no uniqueness check.

Impact: A single user can hold the button and inflate any review's helpfulness without bound, manipulating product review ranking.

LK-H09 — Three different totals for the same cart HIGH

`cart.service.ts:141–142 · order.service.ts:46–47 · checkout.component.ts:101`

The backend cart taxes the *undiscounted* subtotal and applies free shipping at `subtotal >= 999`; order creation taxes `(subtotal − discount)` and applies free shipping at `(subtotal − discount) >= 999`; and the checkout review screen computes `subtotal − discount + shipping` with **no tax term at all**.

Impact: The customer is shown a total roughly 18% below what is actually charged. Guaranteed billing disputes on every online order.

LK-H10 — `free_shipping` coupons do nothing HIGH

`coupon.service.ts:91 returns 0; order.service.ts:46 never inspects the coupon type`

Impact: A coupon type offered in the admin UI silently has zero effect. Customers redeem it, see no benefit, and the ₹99 shipping is still charged.

LK-H11 — Percentage coupons above 100% produce negative totals HIGH

`coupon.service.ts:41–62 (no bound on discount_value ) and :96 (percentage branch uncapped)`

The `fixed` branch is capped with `Math.min(value, subtotal)`; the percentage branch is not.

Impact: A coupon created with `discount_value = 500` yields a discount five times the subtotal, producing a negative `total_amount` that flows into Razorpay and the revenue reports.

LK-H12 — Abandoned online checkouts hold stock forever HIGH

`backend/src/services/order.service.ts:86–96 and :109–111`

Stock is decremented at order creation, but for `razorpay` / `online` orders the cart is deliberately not cleared and payment may never arrive. Release depends entirely on the browser calling `abort-payment`; there is no expiry job or reservation timeout.

Impact: Every closed tab, crashed browser or network drop during payment permanently removes inventory. Stock silently drains to zero and real customers see "out of stock" on available goods.

LK-H13 — Order abort hard-deletes records outside a transaction HIGH

`backend/src/services/order.service.ts:353–375`

`abortOnlineOrder` restores stock, then deletes `order_items`, then deletes the order — three separate statements with no transaction, no status check, and no `payment_method` check.

Impact: A failure between statements leaves stock restored on an order that still exists, so a retry double-restores. The audit trail is destroyed outright. It also accepts COD orders, letting a customer erase their own order record. The frontend calls this on Razorpay modal dismiss — so a payment that actually succeeded but whose modal glitched destroys the order while the money is taken.

LK-H14 — Cancelling twice restores stock twice HIGH

`backend/src/services/order.service.ts:293–302`

The `status === 'cancelled'` restore block never checks whether the order was already cancelled, and `updateStatus` permits any transition.

Impact: An admin double-click inflates inventory by the full order quantity. Additionally, `returned` and `refunded` never restore stock at all, and the Shiprocket auto-sync path (:377–403) sets `cancelled` while bypassing the restore logic entirely — three inconsistent paths.

LK-H15 — Suspended users keep working tokens for 8 hours HIGH

`backend/src/middleware/auth.middleware.ts:22–39`

`authenticate` verifies the JWT signature only; it never re-checks that the user still exists, is not suspended, or still holds the role encoded in the token.

Impact: Banning or demoting a user has no effect until their token expires. A demoted admin retains full admin API access for up to 8 hours.

LK-H16 — Email verification can un-suspend an account HIGH

`backend/src/services/auth.service.ts:153–164`

`verifyEmail()` sets `status = "active"` unconditionally on any valid verification token, with no check of the current status.

Impact: A suspended user who kept their original verification email can replay the link and reactivate their own account, defeating moderation. Note also that registration already sets `status='active'` (:29) and login never checks `email_verified_at` — so verification is decorative.

LK-H17 — Password-reset and verification tokens stored in plaintext HIGH

`auth.service.ts:125–128 (reset), :25–33 (verification); looked up with WHERE token = ?`

Impact: Any read access to the `users` table — SQL injection elsewhere, a backup leak, an over-privileged support account — allows an attacker to take over any account by using the stored reset token directly. These should be stored as hashes.

LK-H18 — Uncapped pagination on public endpoints HIGH

`review.controller.ts:12 (public), order.controller.ts:47,98, coupon.controller.ts:20, admin.controller.ts:20, blog/contact/newsletter routes`

Some controllers cap with `Math.min(Number(limit), 100)` (product, admin products, hair-solutions) but the majority pass `Number(limit)` straight through. `db.paginate()` applies no cap of its own — while the unused `helpers.paginate()` that *does* cap at 100 sits dead in the codebase.

Impact: `GET /reviews/product/1?limit=99999999` is an unauthenticated memory-exhaustion vector. Also `page=0` or `page=-1` produces a negative OFFSET and `limit=abc` produces `LIMIT NaN`, both surfacing as raw 500s.

LK-H19 — Raw database errors returned to clients HIGH

`backend/src/middleware/error.middleware.ts:29–42`

Non-`AppError` exceptions fall through with `message: err.message` in all environments; only the stack is gated behind `NODE_ENV === 'development'`.

Impact: MySQL messages such as `ER_DUP_ENTRY: Duplicate entry 'x@y.com' for key 'users.email'` reach the browser, disclosing schema structure and enabling user enumeration. Multer size errors and malformed-JSON errors also surface as 500s instead of 413/400.

LK-H20 — Admin panel is gated only on client-side state HIGH

`frontend/src/app/core/guards/admin.guard.ts:5–16 → auth.service.ts:138–152`

`adminGuard` reads `isAdmin()`, which is computed from the `lk_user` JSON object restored from `localStorage` and never re-validated against the server. Three `console.log` statements (lines 8, 11, 14) additionally leak every access decision and URL to the browser console in production.

Impact: Editing `localStorage['lk_user']` to `role: "admin"` renders the full admin UI tree. Real protection rests entirely on the backend's `authorize()` middleware — which is correctly applied on the routes audited, so this is UI exposure rather than data compromise. It remains a reconnaissance aid and must not be relied on as a boundary.

LK-H21 — Tokens and user identity stored in localStorage HIGH

`frontend/.../auth.service.ts:131–135 (lk_token, lk_refresh_token, lk_user)`

Impact: Any XSS anywhere in the application exfiltrates both the access and the 30-day refresh token. Severity is amplified by helmet running with `contentSecurityPolicy: false` (app.ts:38) and by the absence of TLS (LK-C08). httpOnly cookies would remove this class of risk.

LK-H22 — Anyone can unsubscribe anyone from the newsletter HIGH

`backend/src/routes/newsletter.routes.ts (POST /unsubscribe)`

The endpoint accepts a bare email address with no signed token, no ownership proof and no rate limit.

Impact: Trivial mass-unsubscribe of the entire marketing list by iterating known addresses.

LK-H23 — Unpublished hair solutions are publicly readable HIGH

`backend/src/routes/hair-solution.routes.ts (GET /:slug)`

The handler runs `SELECT * FROM hair_solutions WHERE slug = ?` with no `status = 'active'` filter, while the sibling `/wigs` and `/patches` endpoints both filter correctly.

Impact: Draft and deactivated products — including internal pricing fields — are exposed to anyone who guesses or scrapes a slug.

LK-H24 — Rejected cart additions are silently added locally HIGH

`frontend/src/app/core/services/cart.service.ts:75–78`

`addItem()`'s `catchError` handler calls `addToLocal(item)`, so a server rejection such as "Only 2 units available" results in the item appearing in the cart anyway.

Impact: Client and server carts diverge with no error shown. The customer proceeds to checkout with phantom items that the server does not have, and the order fails or silently differs.

LK-H25 — Guest cart is discarded on login HIGH

`frontend/src/app/core/services/cart.service.ts:157–182`

`syncFromServer()` overwrites `_items` wholesale. No merge of the anonymous `lk_cart` into the server cart is performed anywhere.

Impact: A guest who fills a cart and then logs in to check out loses everything — a direct and very common conversion loss.

LK-H26 — Deleting a sold product raises a raw FK error HIGH

`backend/src/services/product.service.ts:350–370; schema.sql:312 (ON DELETE RESTRICT)`

`delete()` issues a hard `DELETE FROM products`. Any product referenced by `order_items` is blocked by the foreign key.

Impact: Admins can never delete a product that has ever been ordered, and receive an unexplained 500 with a raw MySQL message instead of a clear "archive instead" response. Image files are also orphaned on disk when images are removed via `update()`.

LK-H27 — Cart quantity is never validated HIGH

`backend/src/controllers/cart.controller.ts:19,28 → services/cart.service.ts:35–79`

`Number(quantity)` is passed through with no integer, sign or upper-bound check. A negative value passes `availableStock < quantity`, and `Number("abc")` yields `NaN`, which also passes because every comparison against `NaN` is false.

Impact: Negative quantities corrupt cart subtotals and can reduce line quantities on merge; `NaN` reaches MySQL and produces a 500. There is also no maximum, so a single line item can be set to an arbitrary quantity.

Medium Severity Findings

Transactions, Concurrency & Data Integrity

LK-M01 — Wishlist counter drifts permanently MEDIUM

`wishlist.service.ts:32,36 (toggle adjusts) vs :41–43 (remove) and :50–52 (clear) — neither adjusts`

`products.wishlist_count` is incremented and decremented by `toggle()` but ignored by `remove()` and `clear()`, and none of it runs in a transaction.

Impact: The counter only ever inflates, permanently misreporting product popularity. `toggle()` also races against the `UNIQUE(user_id, product_id)` constraint, producing 500s on concurrent clicks.

LK-M02 — Cart creation races on the unique constraint MEDIUM

`cart.service.ts:5–12; schema.sql:210 (user_id ... UNIQUE)`

`getOrCreate()` performs SELECT-then-INSERT with no upsert. Concurrent first-time requests (the app fires cart and wishlist loads together) both see null and both insert.

Impact: Intermittent `ER_DUP_ENTRY` 500s on first page load for new users.

LK-M03 — Address default flag can be lost MEDIUM

`user.service.ts:71–83, :97–113, :121–126`

Every default-address path clears all flags and then sets one, in two unwrapped statements.

Impact: A failure between the two leaves the user with no default address.

LK-M04 — Order status updates are not atomic MEDIUM

`order.service.ts:261–303`

Status update, history insert and stock restoration run as independent queries; the history failure is caught and written to `console.error` rather than the logger.

Impact: Partial state on failure, and a silently incomplete audit trail.

LK-M05 — Payment capture is not atomic and not idempotent MEDIUM

`payment.service.ts:82–105`

Order update, transaction log and cart clear are three separate statements with no transaction and no already-paid guard (unlike `createRazorpayOrder`, which does check at :30).

Impact: Duplicate `payment_transactions` rows and inconsistent state on retry.

LK-M06 — Payment ledger records the wrong amount MEDIUM

`payment.service.ts:95–99 (SELECT ... total_amount FROM orders)`

The insert always logs the full order total, even when only an advance was captured.

Impact: Financial reconciliation data is wrong for every partial payment.

LK-M07 — Order numbers and SKUs collide without recovery MEDIUM

`order.service.ts:421–428; helpers.ts:10–18; both use Math.random() against UNIQUE columns`

Impact: A collision surfaces as a raw duplicate-key 500 and the customer's order is lost, with no retry logic. `Math.random()` also makes order numbers guessable.

LK-M08 — Slug generation races and can produce an empty slug MEDIUM

`product.service.ts:380–391; helpers.ts:1–8`

`generateSlug` strips everything outside `[\w\s-]`, so a fully non-Latin product name (Hindi, for example) reduces to an empty string. `generateUniqueSlug` then loops with one query per attempt and offers no protection against two concurrent creates choosing the same slug.

Impact: Broken product URLs for non-English names; duplicate-key 500s under concurrency; unbounded query loops.

LK-M09 — Coupon usage counters disagree with each other MEDIUM

`coupon.service.ts:20–26 vs order.service.ts:101–106`

Per-user usage is derived by counting non-cancelled orders, while the global `usage_count` is a standalone counter that is incremented on order creation and never decremented on cancellation.

Impact: Cancelled orders permanently consume global coupon capacity, so a campaign silently exhausts early.

API Contract & Validation

LK-M10 — Cart and order coupon validation are divergent duplicates MEDIUM

`cart.service.ts:113–134 duplicates coupon.service.ts:5–30, minus the min_order_amount check`

Impact: The cart accepts coupons that order creation will silently reject, feeding directly into LK-C12.

LK-M11 — Product status validated on one endpoint, not the other MEDIUM

`product.service.ts:344–348 validates; :88–102 (patch) does not`

Impact: `PATCH /admin/products/:productId` accepts any string as status, making a product invisible everywhere with no error. `stock_quantity` is likewise unbounded there and in `admin.service.updateInventory` (:119–124).

LK-M12 — Admin can rewind a delivered order to "shipped" MEDIUM

`order.service.ts:331–336`

`updateTracking` forces `status = 'shipped'` unconditionally and writes no status-history entry.

Impact: Editing a tracking number on a delivered order silently reverts its state and leaves no audit record.

LK-M13 — No order state machine MEDIUM

`order.service.ts:261–268`

Any status in `order_statuses` is accepted from any current status; only customer-initiated cancellation restricts source states (:345).

Impact: Illegal transitions such as `delivered → pending` are permitted by API, by bug, or by mistake.

LK-M14 — Audit-log entries are editable and deletable MEDIUM

`order.service.ts:405–419 (updateStatusHistory, deleteStatusHistory)`

Both accept a bare history id with no order-ownership check and no logging of the change itself.

Impact: An admin can rewrite or erase order history, defeating the purpose of the audit trail.

LK-M15 — Review creation does not verify the product exists MEDIUM

`review.service.ts:34–56`

Impact: An invalid `product_id` produces a raw FK 500. The verified-purchase check also matches `o.status IN ('delivered','completed')`, but `'completed'` is not among the seeded statuses — dead condition.

LK-M16 — No input validation library in use MEDIUM

`express-validator is a declared dependency with zero references in src/`

All validation is ad-hoc presence checking scattered through controllers.

Impact: No email, phone or pincode format validation anywhere; password policy is a bare 8-character minimum (user.service.ts:43). Required address fields (`phone`, `city`, `pincode`) are unchecked and reach MySQL as `undefined`, producing 500s.

LK-M17 — Registration races the unique email constraint MEDIUM

`auth.service.ts:19–33`

Impact: Concurrent signups with the same address return a raw duplicate-key 500 instead of the intended clean 409.

LK-M18 — Refresh-token failures are indistinguishable from outages MEDIUM

`auth.service.ts:106–108 (bare catch swallowing everything)`

Impact: A database outage is reported to the user as "invalid or expired refresh token", forcing spurious logouts and hiding real incidents from logs.

LK-M19 — Login accepts every non-suspended status MEDIUM

`auth.service.ts:58`

Impact: Only `'suspended'` is blocked; any other status value, including `'inactive'` or a typo, permits login.

LK-M20 — Profile fields cannot be cleared MEDIUM

`user.service.ts:22–25 (truthiness guards)`

Impact: Submitting an empty `last_name` or `phone` is treated as "no change", so a user can never remove a value once set.

LK-M21 — Admin cannot remove all product images MEDIUM

`product.service.ts:304–306`

When `existing_images` is `"[]"` and no new files are uploaded, `allImages` is empty and the code falls back to the previous `product.images`.

Impact: Deleting every image silently restores the old set. Removed images are also never unlinked from disk.

LK-M22 — Product slug and SKU are immutable after creation MEDIUM

`product.service.ts:308–339 (neither column appears in the UPDATE)`

Impact: Renaming a product leaves a stale SEO URL with no way to correct it.

LK-M23 — No price sanity checks on products MEDIUM

`product.service.ts:266, 325` (`Number(data.price)` unchecked; `Number(data.mrp) || price`)

Impact: A missing price becomes `NaN`; `price > mrp` is permitted, producing negative discount percentages in the storefront.

LK-M24 — Shipping method selection is ignored MEDIUM

`checkout.component.ts:152` sends `shipping_method`; `order.controller.ts:11` never reads it

Impact: The shipping choice presented to the customer is purely decorative; cost is hardcoded server-side as ₹99 / free above ₹999.

LK-M25 — Business rules hardcoded in application code MEDIUM

`order.service.ts:46–47` (₹99 shipping, ₹999 threshold, flat 18% GST)

Impact: Tax and shipping cannot be changed without a redeploy, a single GST slab is applied to all products regardless of category, and `Math.round` on the whole tax introduces rounding drift.

Availability & Performance

LK-M26 — External API call inside a page-load read path MEDIUM

`order.service.ts:119–183`

`getById` calls Shiprocket synchronously whenever an order is more than 15 minutes stale, using a dynamic `import()` on every invocation and with no visible timeout.

Impact: Order-detail latency and availability are coupled to a third party. This belongs in a background job.

LK-M27 — DDL executed on hot request paths MEDIUM

`order.service.ts:271–282` and `:307–317` (`CREATE TABLE IF NOT EXISTS` per call)

Impact: Every status update and every history read issues a DDL statement. This also forces the production database user to hold CREATE privileges permanently.

LK-M28 — Writes on public read endpoints MEDIUM

`product.service.ts:207` (`view_count`), `blog.routes.ts` (`view_count`)

Both are awaited inline on every GET, with no batching or deduplication.

Impact: Row-lock contention on popular products, added latency on every product page, and inflated counts from bots.

LK-M29 — Search does a full table scan MEDIUM

`product.service.ts:129, :220` (`LIKE '%term%'` across name/description/tags/ingredients)

No FULLTEXT index exists; `products` carries only `idx_products_status`.

Impact: Search degrades linearly with catalogue size. User-supplied `%` and `_` are also unescaped and act as wildcards.

LK-M30 — Dashboard aggregates are uncached and index-hostile MEDIUM

`admin.service.ts:5–75` (eight aggregate queries per load); `:13,:20,:28` wrap columns in `DATE()` / `YEARWEEK()`

Impact: Wrapping the indexed column in a function prevents index use, forcing full scans of `orders` and `users` on every dashboard view.

LK-M31 — Connection pool has no timeouts and an unlimited queue MEDIUM

`utils/database.ts:9–22` (`queueLimit: 0`, no `connectTimeout`)

Impact: Under load, requests queue indefinitely rather than failing fast, converting a slow database into an unbounded latency pile-up.

LK-M32 — Graceful shutdown leaks the pool and can hang MEDIUM

`server.ts:124–133`

`shutdown()` closes the HTTP server but never ends the MySQL pool, has no forced-exit timeout, and there are no `unhandledRejection` / `uncaughtException` handlers.

Impact: Deploys can hang on open connections; unhandled rejections will terminate the process on modern Node with no logging.

LK-M33 — Migrations run on every boot MEDIUM

`server.ts:7–101`

Imperative `CREATE TABLE` / `ALTER TABLE` statements execute at startup, in parallel with `schema.sql` and a `migrations/` directory — three competing schema mechanisms.

Impact: Multi-instance deploys race on DDL; there is no migration ledger, so schema state is not reproducible. This is how the `avatar_url` gap (LK-C09) went unnoticed.

Configuration & Deployment

LK-M34 — Content Security Policy disabled MEDIUM

`app.ts:36–39` (`contentSecurityPolicy: false`)

Impact: Removes the primary mitigation against XSS, which matters most given tokens live in `localStorage` (LK-H21).

LK-M35 — Uploads directory loses its security headers MEDIUM

`nginx.conf:36–42 vs :70–75`

The `/uploads/` location defines its own `add_header`, and nginx does not inherit parent `add_header` directives into a location that declares any of its own. `X-Content-Type-Options: nosniff` and `X-Frame-Options` are therefore absent on user-uploaded content, which is additionally served with `Access-Control-Allow-Origin: *`.

Impact: Any non-image that reaches the uploads directory can be sniffed and executed as active content in the user's origin.

LK-M36 — Upload filter trusts the client-declared MIME type MEDIUM

`middleware/upload.middleware.ts:8–14`

Only `file.mimetype` is checked — no extension check and no magic-byte inspection.

Impact: Largely mitigated in practice because sharp re-encodes every upload to WebP, which destroys polyglot payloads. Still worth hardening, and it means an upload that bypasses the sharp path would be unprotected.

LK-M37 — Product uploads fail on a fresh deployment MEDIUM

`product.service.ts:393–403 (no mkdirSync) vs media.routes.ts (creates the directory)`

Impact: If `uploads/products/` does not already exist, sharp's `.toFile()` throws and product image upload fails until the directory is created by hand.

LK-M38 — Static uploads path depends on the working directory MEDIUM

`app.ts:77` (`express.static('uploads')` — relative) vs `:119` (`process.cwd()` — absolute)

Impact: Starting the process from any other directory silently breaks image serving.

LK-M39 — SPA fallback masks missing assets as HTTP 200 MEDIUM

`app.ts:123–131`

The `app.get('*')` handler returns `index.html` for any unmatched non-API GET, including missing files under `/uploads` that `express.static` passed through.

Impact: Missing images resolve to an HTML document with status 200, breaking client-side error handling and confusing search-engine crawlers.

LK-M40 — Same router mounted on two paths MEDIUM

`app.ts:102–103` (`/users` and `/account` both serve `userRoutes`)

Impact: Duplicated public surface that must be secured, rate-limited and monitored twice; the frontend uses `/account` while the canonical path appears to be `/users`.

LK-M41 — No secret validation at startup MEDIUM

`utils/config.ts:57–62` (`'change-this-in-production'` fallbacks)

Impact: The server boots happily with default JWT secrets and never warns. This is precisely the failure mode LK-C07 exploits. A production start should refuse to run on default secrets.

Low Severity & Code Quality

Broken or Dead Tooling

LK-L01 — Both test scripts are non-functional; zero tests exist LOW

`backend/package.json:10` (`jest not installed`) · `frontend/package.json:12` (`no karma/jasmine`)

Verified: no `.spec.ts` or `.test.ts` file exists anywhere in the repository.

Impact: There is no automated safety net. Every finding in this report would have to be caught by hand. This is the single highest-leverage gap for preventing regressions.

LK-L02 — Database npm scripts point at non-existent files LOW

`backend/package.json:11–12` → `dist/utils/migrate.js`, `dist/utils/seed.js`

Neither `migrate.ts` nor `seed.ts` exists in `src/utils/` (only `seed_demo_data.ts`).

Impact: `npm run db:migrate` and `db:seed` fail immediately.

LK-L03 — TypeScript path aliases reference missing directories LOW

`backend/tsconfig.json` (`@repositories/*`, `@models/*`)

Neither directory exists, no import uses the aliases, and no runtime path resolver is configured.

Impact: Adopting these aliases later would compile but fail at runtime.

LK-L04 — Unused and misleading dependencies LOW

`backend/package.json` — `sequelize` (zero references), `express-validator` (zero references)

Impact: `sequelize` implies an ORM that is not used anywhere; all access is raw `mysql2`. Both inflate install size and audit surface. `multer@1.4.5-lts.1` and `sharp@0.32.6` are both old enough to warrant a security audit.

LK-L05 — Dead security helpers give false assurance LOW

`utils/helpers.ts:28–35` (`sanitizeHtml`), `:37–41` (`paginate`), `:20–26` (`formatPrice`)

All three are defined and never imported anywhere — verified by search.

Impact: `sanitizeHtml` suggests output is escaped when nothing calls it, and `paginate` contains the `limit` cap that LK-H18 needs.

LK-L06 — Development scripts committed to the repository LOW

`backend/check_admins.js, check_customer.js, gen_tokens.js, seed_demo_solutions.js (748 lines)`

Impact: Beyond the token-forging risk of LK-C07, these ship database access patterns and demo data into production.

LK-L07 — Build artifacts committed to version control LOW

`backend/ui/` — 160 tracked files, roughly 14 MB

Impact: Compiled Angular output is tracked despite `.gitignore` excluding `dist/`. Every deploy produces a large, unreviewable diff and the artifact can drift from its source.

LK-L08 — No CI pipeline present LOW

`.github/workflows/ is absent, though commit c4b1cd4 references deploy.yml`

Impact: The deployment workflow referenced in history is untracked, so deploys are unreproducible from the repository alone.

Frontend Quality

LK-L09 — No HTTP interceptor anywhere LOW

`Verified: zero matches for HttpInterceptor / withInterceptors across the app`

Auth headers are attached by hand in all six `ApiService` methods.

Impact: No central 401 handling, no automatic logout, no refresh trigger, no correlation IDs, no timeout or retry policy. This is why LK-H03's broken refresh URL was never noticed.

LK-L10 — Logout never notifies the server LOW

`auth.service.ts:114–123 — clears localStorage but never calls POST /auth/logout`

Impact: The refresh-token row survives in the database indefinitely, so "logging out" does not invalidate the server-side session.

LK-L11 — Cart mutations silently swallow server errors LOW

`cart.service.ts:90, :106 ( catchError(() => of(null)) )`

`WishlistService` implements proper rollback; the cart does not.

Impact: Quantity and removal failures leave stale UI with no user feedback.

LK-L12 — Cart summary has two sources of truth LOW

`cart.service.ts:49 (client-computed subtotal ) vs :173–180 (server _summary ); discount hardcoded to 0 at :177`

Impact: Checkout reads the client-computed value (checkout.component.ts:101), so any divergence from the server is displayed to the customer. The applied coupon never reaches the summary.

LK-L13 — Guest cart item IDs are timestamps LOW

`cart.service.ts:152 ( id: Date.now() )`

Impact: Two items added within the same millisecond collide, and the synthetic IDs are structurally indistinguishable from server IDs.

LK-L14 — SEO service bypasses the DOCUMENT token LOW

`core/services/seo.service.ts ( setCanonical , injectSchema use global document )`

Impact: Inconsistent with the platform-guarded pattern used everywhere else, and fragile under SSR and unit testing.

LK-L15 — Every route is server-rendered on every request LOW

`app.routes.server.ts — a single ** route with RenderMode.Server`

Impact: Fully static pages (about, terms, privacy, refund, shipping) are re-rendered per request instead of being prerendered, wasting CPU and inflating TTFB.

LK-L16 — Admin components diverge from the project's own conventions LOW

`All 15 admin components use inline templates and styles; the other 36 split into .html/.scss`

`admin-product-edit.component.ts` is 924 lines of combined template, logic and styles.

Impact: The most security-sensitive and mutation-heavy area of the app is also the hardest to review.

LK-L17 — Substantial duplication between wig and patch admin screens LOW

`admin-hair-wig-edit (530 lines) vs admin-hair-patch-edit (518 lines) — roughly a 150-line diff`

Impact: Every fix must be applied twice; the pair will drift.

LK-L18 — Weak typing concentrated in the admin area LOW

`118 uses of` `any` `across 13 admin files`

Impact: `strictTemplates` and `strict` are enabled project-wide, so this is a deliberate opt-out precisely where correctness matters most.

LK-L19 — Debug logging left in shipped code LOW

`admin.guard.ts:8,11,14 · admin-order-detail · admin-settings · account/order-detail (7 total)`

Impact: The guard entries log authentication decisions and navigation targets to the production console.

LK-L20 — Redundant cart clearing after checkout LOW

`checkout.component.ts:177 and :249 call` `clearCart()`, `which the server already did`

Impact: An unnecessary `DELETE /cart` round trip after every order; harmless but indicates unclear ownership of cart lifecycle.

Backend Quality & Observability

LK-L21 — Errors silently swallowed in service code LOW

`order.service.ts:326 (returns` `[]`), `:398 (bare catch), :288 (` `console.error`), `auth.service.ts:43,131`

Impact: Failures never reach the winston logger, so they are invisible in production monitoring. Status history silently returns empty on any error.

LK-L22 — Audit logging is incomplete LOW

`utils/activity-logger.ts — called only from auth (admin login), category, product and order-status paths`

Not logged: refunds, payment verification, coupon CRUD, review moderation, inventory updates, customer changes, hair-solution CRUD, blog CRUD, and status-history edits and deletions.

Impact: The highest-risk financial actions are precisely the ones with no audit record. Calls are also fire-and-forget, so failures are lost.

LK-L23 — Inconsistent logging channels LOW

`console.error` `vs the winston` `logger` `mixed across auth, order and product services`

Impact: Log aggregation misses everything written to `console`.

LK-L24 — Redundant dynamic imports LOW

`order.controller.ts:250 dynamically imports logActivity already imported at :4; order.service.ts:141,169 and order.controller.ts:17 import db/logger dynamically`

Impact: Per-call module resolution overhead and inconsistent style with no benefit, since CommonJS caches the module anyway.

LK-L25 — Non-timing-safe signature comparison LOW

`payment.service.ts:76 and :118 use !== rather than crypto.timingSafeEqual`

Impact: Theoretical timing side channel on HMAC verification; low practical risk over a network, but trivially fixable.

LK-L26 — Reviewer PII exposed on a public endpoint LOW

`review.service.ts:9–10 returns u.last_name in full and SUBSTRING(u.email,1,3)`

Impact: Partial email prefixes and full surnames of customers are published on product pages with no opt-in.

LK-L27 — Product rating becomes NULL when the last review is removed LOW

`review.service.ts:111–119 (AVG over an empty set)`

Impact: `rating_avg` is set to NULL rather than the 0.00 the schema defaults to, and the frontend expects a number.

LK-L28 — Fabricated profit figures on the admin dashboard LOW

`admin.service.ts:42–44 –` `COALESCE(p.cost_price, oi.unit_price * 0.6)`

When `cost_price` is NULL the query invents a 60% cost assumption and presents the result as profit.

Impact: Business decisions made on silently synthetic numbers, with nothing in the UI indicating the value is estimated.

LK-L29 — Revenue reporting is not refund-aware LOW

`admin.service.ts:17–24, :65–75 (WHERE payment_status = 'paid')`

Refunding an order changes its `payment_status`, so it silently disappears from historical revenue.

Impact: Past reports change retroactively; revenue also includes tax and shipping.

LK-L30 — Product statistics mislabelled LOW

`product.service.ts:80 –` `stats.total = active + inactive + draft`, excluding archived

Impact: The admin "total products" figure never matches the actual row count.

LK-L31 — Cart item count reports line items, not units LOW

`cart.service.ts:144 (item_count: items.length)`

Impact: The header badge shows 2 when the cart holds 10 units. The frontend's own `count` computed signal (cart.service.ts:48) sums quantities correctly, so the two disagree.

LK-L32 — Contact and newsletter forms are unprotected LOW

`contact.routes.ts (POST /), newsletter.routes.ts (POST /subscribe)`

No CAPTCHA, no dedicated rate limit, no email format validation, no double opt-in. Contact submissions trigger an outbound email per request. The schema's `ip_address` and `user_agent` columns are never populated.

Impact: Spam and outbound email amplification, with no forensic data captured. Subscribe also compares an untrimmed address but inserts a trimmed one, so a leading space bypasses the duplicate check and then trips the unique constraint as a 500.

LK-L33 — Sensitive fields returned by `SELECT *` LOW

`order.service.ts:123–131 (o.* includes razorpay_signature), coupon.service.ts:33,38; hair-solution.routes.ts`

Impact: Payment signatures and internal fields are returned to clients that have no use for them; adding a column silently widens the API response.

LK-L34 — Deleting an order status orphans referencing orders LOW

`admin.routes.ts (DELETE /order-statuses/:id); orders.status is a VARCHAR with no foreign key`

Impact: Orders retain a status slug that no longer resolves, so the LEFT JOIN in `getById` yields a null status name and the UI renders blank. The same applies to hard-deleted coupon codes.

LK-L35 — Stored cart unit price is dead data LOW

`cart_items.unit_price written at cart.service.ts:71, never read; getCart joins live p.price`

Impact: Harmless and in fact safer — live pricing prevents stale-price attacks — but the column silently goes stale and misleads readers. `carts.coupon_code` is likewise never written.

LK-L36 — Prepared-statement placeholders in LIMIT/OFFSET LOW

`utils/database.ts:71–72 (LIMIT ? OFFSET ? via pool.execute)`

Impact: mysql2 is sensitive to the argument type here; a string-typed limit produces `Incorrect arguments to mysqld_stmt_execute`. This is the mechanism by which LK-H18's unvalidated `limit` surfaces as a 500.

LK-L37 — Rollback failure masks the original error LOW

`utils/database.ts:52–54`

Impact: If `conn.rollback()` throws, the underlying business error is replaced by a connection error, obscuring the real cause.

LK-L38 — Missing indexes on frequently filtered columns LOW

`schema.sql – refresh_tokens.token, reviews.status, products.category_id filters, blog_posts.status`

`refresh_tokens` is looked up by `WHERE token = ?` on a `VARCHAR(500)` with an index only on `user_id`.

Impact: Every token refresh scans the table; review moderation and blog listings scan as the tables grow.

Recommended Remediation Order

#	Action	Addresses	Why first
1	Rotate Razorpay secret, Shiprocket password and `JWT_SECRET`; purge secrets from git history; delete `gen_tokens.js`	C06, C07	Live credentials are already exposed; every hour counts
2	Enable TLS in nginx with an HTTP→HTTPS redirect	C08	Everything else is interceptable until this is done
3	Derive the charge amount server-side; bind `razorpay_order_id` to the order row; add an idempotency guard	C01, C02, M05	Closes direct, unauthenticated revenue loss
4	Mount `express.raw()` for the webhook path before the global `express.json()`; require the webhook secret at boot	C03, C04	Both failure modes must be fixed together to work at all
5	Set `app.set('trust proxy', 1)`	C11	One line restores every rate limiter in the system
6	Add the `avatar_url` column, or remove it from the query	C09	A user-facing endpoint fails 100% of the time
7	Send `coupon_code` from checkout; align tax and shipping across cart, order and UI	C12, H09, H10, H11	Customers are currently charged more than they are shown
8	Move the stock check inside the transaction with a `WHERE stock_quantity >= ?` guard; add a reservation timeout	C10, H12	Prevents overselling and silent inventory drain
9	Bound refunds by the remaining refundable balance; log every refund	C05, L22	Financial control with no current audit trail
10	Contain the media upload `folder` parameter to the uploads root	H01	Arbitrary file write
11	Populate `razorpayKeyId` from the API response; fix the `/auth/refresh-token` URL	H02, H03	Two one-line fixes that unbreak production checkout and sessions
12	Add an HTTP interceptor for auth headers, 401 handling and refresh	L09, L10, H21	Structural fix that prevents a whole class of recurrence
13	Introduce `express-validator` schemas; cap `limit` centrally in `db.paginate`	H18, H27, M16	Already a dependency; closes the DoS and 500-noise surface
14	Sanitise error responses in production; keep detail server-side	H19	Stops schema and user enumeration leakage
15	Set up jest with tests covering the payment and order paths first	L01	Without this, every fix above can silently regress

What Is Working Well

- **No SQL injection was found.** Every query uses parameter binding, and the sort and order inputs in `product.service.ts:110–111` are correctly allow-listed rather than interpolated.
- **Password handling is solid** — bcrypt at cost 12, correct compare-then-issue ordering, and refresh tokens persisted and rotated in the database rather than being stateless.
- **Backend authorization is applied consistently.** Every admin route audited carries `authenticate` plus `authorize('super_admin','admin')`, so LK-H20 is UI exposure rather than data compromise.
- **Cart pricing is read live from the products table** rather than trusted from the client, which eliminates an entire family of price-tampering attacks.
- **Image uploads are re-encoded through sharp**, which neutralises polyglot payloads and substantially mitigates the weak MIME check.
- **SSR platform guards are applied carefully** across navigation, cart and auth services — 14 files use browser APIs and nearly all are correctly guarded.
- **Routing is clean** — fully lazy-loaded feature areas, correct specific-before-parameterised ordering, and no shadowed routes were found.
- **Order creation uses a real transaction** (`order.service.ts:54–114`), which is the right foundation; the fix for LK-C10 is to widen its boundary, not to introduce one.
